

NAVIS: Web-Native Interactive Visualization of Dynamic Point-Cloud Video

Jashanjot Singh Sidhu*
Concordia University
Canada

Jérémy Ouellette*
Concordia University
Canada

Abdelhak Bentaleb
Concordia University
Canada

Abstract

Volumetric media in the form of dynamic point clouds is increasingly adopted for immersive applications such as virtual reality, telepresence, digital twins, and interactive 3D experiences, yet practical support for browser-native visualization remains limited. Existing solutions often rely on heavyweight native players, offline preprocessing, or specialized plugins, creating a significant barrier to accessibility and deployment at scale. In this demo, we present NAVIS, a fully browser-based volumetric media player that enables interactive playback and exploration of dynamic PLY point-cloud sequences using standard web technologies. NAVIS combines WebGL-based rendering with a parallel web-worker parsing pipeline, buffered playback with adaptive concurrency control, and interactive navigation modes, allowing large volumetric datasets to be visualized smoothly directly within the browser. We instrument NAVIS to expose key performance metrics, including startup delay, parsing latency, and stall duration, and evaluate it across multiple volumetric datasets at 30 fps and 60 fps. Our results show that increasing worker parallelism substantially reduces startup delay and mitigates playback stalls, demonstrating that high-performance volumetric media visualization is feasible on the web without native dependencies. NAVIS provides a practical and extensible foundation for future browser-native immersive media systems.

CCS Concepts

• Information systems → Multimedia streaming.

Keywords

Volumetric media, point clouds, browser-based visualization.

ACM Reference Format:

Jashanjot Singh Sidhu, Jérémy Ouellette, and Abdelhak Bentaleb. 2026. NAVIS: Web-Native Interactive Visualization of Dynamic Point-Cloud Video. In *ACM Multimedia Systems Conference 2026 (MMSys '26)*, April 04–08, 2026, Hong Kong, Hong Kong. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3793853.3798192>

1 Introduction

Multimedia streaming already accounts for a dominant share of global Internet traffic—estimated at over 65% [19]—and this trend continues to accelerate with the emergence of increasingly immersive media formats [2, 18]. Beyond traditional 2D video, recent

years have seen a rapid rise in volumetric media, particularly content represented using point clouds [11] and meshes [21], which enables six degrees of freedom (6DoF) [8] and supports immersive experiences such as virtual reality, telepresence, digital twins, and interactive 3D environments. These representations allow users to freely navigate a scene and observe content from arbitrary viewpoints, significantly enhancing realism and user engagement.

For both researchers and practitioners, the ability to visualize volumetric content is essential. Visualization plays a critical role in evaluating encoding pipelines, analyzing temporal artifacts, validating streaming strategies, and debugging end-to-end systems for immersive media delivery. However, existing visualization solutions exhibit important limitations. Several tools focus on rendering large-scale point clouds [20], but primarily support static content or rely on heavyweight native applications with limited interactivity. More recently, MPEG has introduced support for rendering dynamic PLY sequences [13], enabling playback of time-varying point clouds, yet current solutions lack browser-native support and offer limited flexibility in terms of buffering, interactivity, and performance instrumentation.

To address these limitations, we present NAVIS¹, a fully browser-based volumetric media player designed for interactive visualization of dynamic PLY point-cloud sequences. NAVIS operates entirely within the web environment, requiring no native plugins or preprocessing, and leverages modern web technologies for real-time rendering and parallel parsing. It supports buffered streaming, adaptive concurrency via web workers, interactive navigation modes, and fine-grained performance monitoring, making it suitable for both live demonstrations and research experimentation. Crucially, NAVIS embraces browser-native visualization to maximize accessibility and portability. By eliminating dependencies on VR headsets, native binaries, or platform-specific toolchains, NAVIS enables dynamic volumetric content to be viewed anywhere, on commodity devices, directly through a web browser. This design lowers the barrier to entry for researchers and practitioners, facilitates rapid sharing and reproducibility, and makes volumetric media easier to inspect, debug, and demonstrate at scale. While NAVIS currently operates in a local-first configuration, loading frame-indexed PLY sequences from the client file system, its design explicitly mirrors a streaming-style playback pipeline with buffering, prefetching, and stall handling. This makes NAVIS a practical testbed for studying volumetric media playback behavior and for future integration with networked delivery mechanisms. The NAVIS player demonstration, along with its implementation, is publicly available at [Demo] and [Code]. Our demo paper makes the following key contributions:

- (1) We introduce NAVIS, a fully browser-native volumetric media player that enables interactive visualization of dynamic

*The first two authors contributed equally to this work.



¹NAVIS abbreviates the following full name of the design: web-NAtive interactive VISualization of dynamic point-cloud video

Table 1: Comparison of point cloud visualization systems.  : Feature present,  : Feature absent.

System	Browser-Based	Application-Based	Dynamic PLY	Buffered Playback	Parallel Parsing	6DoF Navigation	Performance Metrics
CloudCompare [7]	�	�	�	�	�	�	�
MeshLab [3]	�	�	�	�	�	�	�
ParaView [12]	�	�	�	�	�	�	�
Open3D [22]	�	�	�	�	�	�	�
Unity / Unreal	�	�	�	�	�	�	�
MPEG 3DG Renderer	�	�	�	�	�	�	�
Potree [20]	�	�	�	�	�	�	�
Three.js PLYLoader [4]	�	�	�	�	�	�	�
NAVIS	�	�	�	�	�	�	�

PLY-based point-cloud sequences without requiring native applications, plugins, or proprietary offline preprocessing (e.g., octree construction or format conversion as required by existing web-based viewers).

- (2) We design and implement a parallel, web-worker-based buffered playback and parsing pipeline with adaptive concurrency control, and rich interactive navigation (orbit and free-fly modes), enabling smooth 6DoF visualization of volumetric content directly in the browser.
- (3) We demonstrate fine-grained, real-time performance instrumentation within NAVIS, exposing metrics such as startup delay, parsing latency, stall duration, and resource usage to support reproducible evaluation and experimentation for volumetric media systems.

The remainder of this paper is organized as follows. We first review related work on point cloud visualization in Section 2. Section 3 then presents the design and architecture of NAVIS, followed by a performance evaluation in Section 4. Section 5 discusses directions for the continued evolution of the system, and Section 6 concludes the paper.

2 Related Work

2.1 Application-Based Point Cloud Viewers

A wide range of mature desktop applications support visualization and analysis of static point clouds. Tools such as CloudCompare [7], MeshLab [3], and ParaView [12] are widely used in research and industry for inspecting large-scale point cloud datasets stored in formats such as PLY, LAS, and E57. These systems provide rich functionality for filtering, measurement, and geometric analysis, but are primarily designed for offline use and static content. While they can efficiently load large datasets, they do not target interactive playback of time-varying point cloud sequences and typically require local installation and platform-specific dependencies. Support for dynamic point cloud visualization is more commonly provided through native applications and rendering frameworks. Game engines such as Unity and Unreal Engine are frequently used to visualize dynamic volumetric content and to demonstrate MPEG point cloud codecs, including V-PCC [1, 9, 10], but rely on heavyweight runtimes and platform-specific builds. Open3D [22] enables dynamic point cloud updates through programmatic frame replacement and is widely used for visualization and algorithm prototyping; however, it operates as a native application framework and does not target media-style playback of dynamic PLY sequences, lacking browser-native deployment, buffered streaming,

and performance instrumentation. More recently, MPEG introduced a dynamic PLY-based renderer as part of the MPEG 3DG reference software [13]. While this renderer supports visualization of dynamic PLY sequences, it follows a file-driven playback model in which all frames must be loaded in advance, and therefore does not support buffered streaming, progressive loading, or interactive concurrency control.

2.2 Web-Based Point Cloud Viewers

Several browser-native solutions exist for visualizing static point clouds. Potree [20] leverages WebGL [17] and hierarchical level-of-detail representations to render large LiDAR datasets efficiently in the browser, but requires offline preprocessing into a custom format and primarily targets static scenes. Similarly, Three.js [4] provides basic loaders such as PLYLoader and PCDLoader for rendering point clouds directly in the browser; however, these loaders are designed for static content and do not support dynamic playback, buffering, or performance-aware streaming. In contrast, support for dynamic point cloud visualization in the browser remains limited. To the best of our knowledge, existing web-based solutions do not provide native support for buffered playback of dynamic PLY sequences, parallel parsing, or fine-grained performance observability required for interactive and reproducible volumetric media experimentation. Existing approaches either focus on static point clouds or rely on native, file-driven renderers for dynamic content. NAVIS addresses these limitations by providing a fully browser-native solution for interactive visualization of dynamic PLY-based volumetric media, supporting buffered streaming, parallel parsing, and explicit exposure of system performance. Table 1 summarizes how NAVIS differs from existing point cloud viewers.

3 NAVIS Design

3.1 System Architecture and Data Model

NAVIS is a browser-based player that treats a sequence of frame-indexed PLY files as a temporal video abstraction, providing a lightweight but explicit execution model for dynamic point cloud playback. Frames are fetched, parsed, buffered, and rendered on demand directly in the browser, without relying on compression formats, native decoders, or specialized streaming infrastructure, allowing all stages of the pipeline to remain observable and controllable. This design emphasizes transparency, determinism, and reproducibility, making NAVIS well suited for interactive demonstrations as well as research-oriented analysis. The player adopts a path-based data access model in which individual frames are retrieved via standard

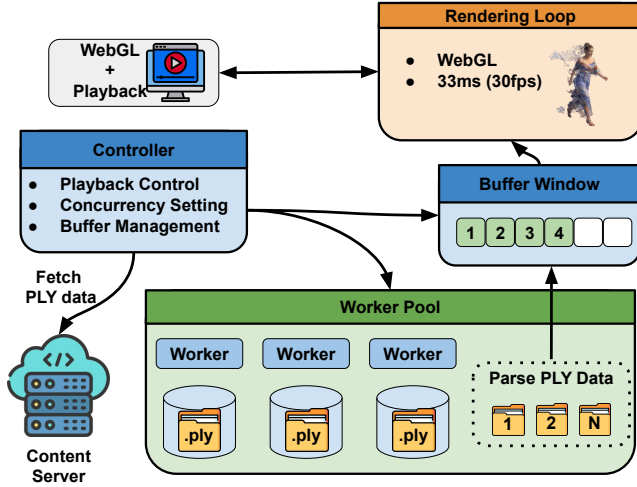


Figure 1: NAVIS player architecture, including the controller, buffering logic, rendering loop, and parallel parsing pipeline.

HTTP requests from a user-specified content path, enabling progressive fetching, explicit buffer window management, and stall detection following a streaming-style playback model. Frames are addressed using deterministic filename-based indexing, allowing consistent random access semantics across datasets. Although the current demo setup typically relies on a lightweight HTTP server, the buffering and scheduling logic is agnostic to deployment and directly extensible to remote, networked delivery scenarios. The overall system architecture, including the central controller, buffering subsystem, rendering loop, and parallel parsing pipeline, is illustrated in Figure 1.

3.2 Playback and Interaction

Playback is coordinated by a central controller responsible for maintaining the current frame index, playback rate, looping state, and synchronization with timeline controls, forming the authoritative system state for both temporal and spatial interaction. Frames are deterministically indexed based on filename padding, enabling frame-accurate seeking and reproducible playback behavior, with support for continuous playback as well as explicit temporal inspection through scrubbing and discrete frame stepping. Rendering is implemented using Three.js on top of WebGL to ensure broad browser and hardware compatibility across commodity devices; integration with WebGPU is planned as part of the NAVIS v2 roadmap to improve GPU efficiency and scalability for larger or denser datasets. Each frame updates the geometry and attributes of a single point-cloud object, avoiding multi-object accumulation and enabling smooth playback as well as immediate visual feedback during seeking and inspection. The rendering pipeline supports adjustable point size, multiple color and attribute modes, optional normalization, and configurable spatial downsampling to accommodate datasets with varying scale and density, along with optional visual aids such as a floor plane and configurable background themes. NAVIS further provides two tightly integrated spatial navigation modes: an orbit mode for object-centric inspection and a fly mode

Table 2: Characteristics of dynamic point cloud sequences.

Content	Frames	FPS	Start Buffer	Avg. Points
RFW [6]	250	30	5	750,100
Longdress [5]	300	30	5	765,821
Robot	250	60	10	1,107,415

for first-person exploration inspired by editor-style controls from modern game engines, allowing seamless exploration of space and time within a unified interaction model, as illustrated in Figure 2.

3.3 Parallel Parsing and Instrumentation

To preserve interactivity under high data rates, computationally intensive PLY parsing is offloaded from the main thread to a bounded pool of web workers that operate concurrently on queued frame requests. Each worker independently parses its assigned frame and returns the result asynchronously using transferable objects to avoid data copying, ensuring minimal overhead on the main rendering thread; frames may therefore complete out of order, but are cached deterministically by frame index and consumed strictly according to playback position. Frame data are stored independently rather than incrementally updating a shared buffer, a design choice that favors simplicity, predictable memory behavior, and robustness to out-of-order completion, and which has proven effective in practice with no observable garbage-collection-induced stutters during interactive use (Figure 4). The degree of parsing parallelism, buffer window size, and prefetch behavior are explicitly configurable at runtime through the user interface, allowing adaptation to available hardware resources and dataset characteristics. Beyond basic playback, NAVIS supports extensive instrumentation, including navigation trace recording that captures camera pose, field of view, and playback state at each frame, enabling detailed analysis of user exploration behavior and serving as input for evaluating view-port prediction, saliency modeling, or behavior-aware adaptation algorithms. Core components such as fetching, buffering, parsing, rendering, and instrumentation are cleanly separated, allowing NAVIS to function both as a standalone visualization tool and as a reusable, extensible foundation for browser-native volumetric media research.

4 NAVIS Performance

To evaluate the performance of NAVIS, we consider three dynamic point-cloud sequences with distinct characteristics: Ready for Winter (RFW), Longdress, and Robot, summarized in Table 2. RFW and Longdress exhibit subtle to moderate body motion at 30 fps with comparable geometric density, whereas Robot features fast-paced articulated motion at 60 fps and substantially higher per-frame point counts. This diversity allows us to assess NAVIS across different motion dynamics, frame rates, and content complexities. We evaluate three key performance metrics: startup delay, stall time, and parse time, while varying the number of Web Worker threads used for PLY parsing (16, 8, and 4 workers). Startup delay measures the time from playback initiation to first rendered frame, stall time captures playback interruptions due to insufficient buffered frames, and parse time corresponds to the cumulative CPU-bound parsing time required to populate the initial start buffer. Our evaluation

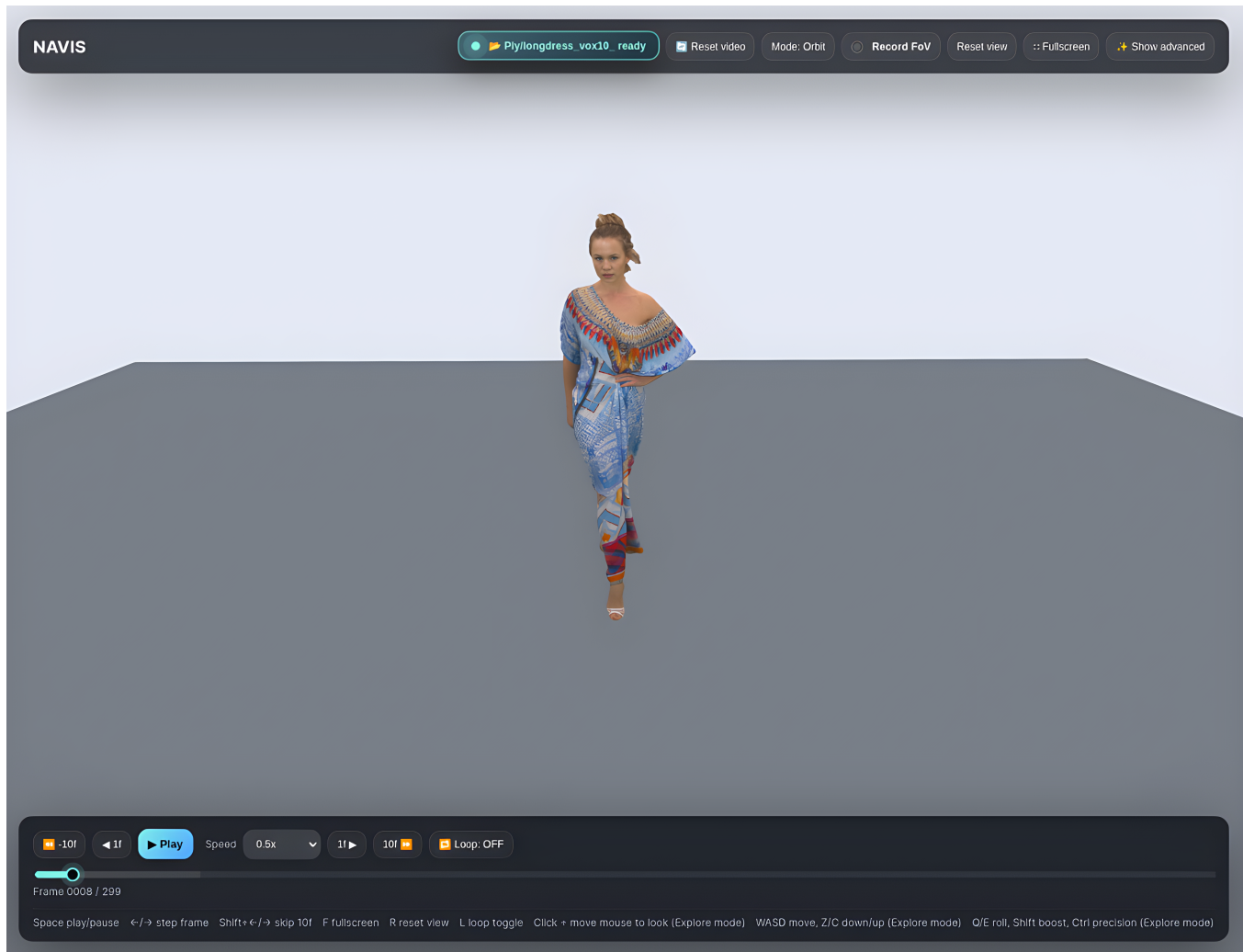


Figure 2: User interface of NAVIS during playback of the *Longdress* sequence from the 8i Labs dynamic point cloud dataset.

focuses specifically on the data availability pipeline—namely frame fetching, parsing, and buffering—implemented using Web Workers. Rendering in NAVIS is handled by a single WebGL render loop targeting the nominal playback rate (30 or 60 fps depending on the content); on the evaluated datasets and hardware, rendering remains stable once frames are available in the buffer, and observed stalls are attributable to buffer underflow rather than GPU saturation. A detailed analysis of GPU-bound rendering limits and WebGPU-based rendering is left for future work.

The results are summarized in Figure 4. For RFW, NAVIS achieves consistently low startup delay and negligible stall time across all worker configurations, with playback remaining stall-free even under reduced parallelism, indicating that moderate-density, low-motion content at 30 fps can be handled robustly with limited parsing resources. Longdress exhibits similar behavior, with slightly increased startup delay and parse time due to higher per-frame point counts, and short stalls appearing only at lower concurrency

levels, while maintaining largely smooth playback under moderate parallelism. In contrast, the Robot sequence presents a more demanding scenario due to its higher frame rate, denser geometry, and fast motion, leading to increased startup delay across all configurations and rapidly growing stall time as parsing concurrency is reduced. These results highlight the critical role of parallel parsing throughput for maintaining buffer stability in high-density, high-FPS volumetric content. Overall, NAVIS scales predictably with both content complexity and available parallelism, achieving low startup latency and near stall-free playback for low- to moderate-motion sequences while clearly exposing how increased content complexity translates into higher parsing requirements for stable playback. All measurements are obtained directly within the browser, underscoring NAVIS as a practical, transparent, and reproducible platform for volumetric media visualization and experimentation.



Figure 3: Advanced parameter panels of NAVIS exposing playback, buffering, navigation, and visualization controls.



Figure 4: Impact of worker parallelism on NAVIS performance for playback of dynamic PLY sequences.

5 Future Directions and Ongoing Development

NAVIS is developed as an open-source system following an iterative and extensible design philosophy. Rather than a fixed artifact, the player is intended to evolve through incremental feature additions,

refinement, and community-driven contributions. Its modular architecture and explicit control interfaces make it easy to integrate with external tools and adapt to new research workflows, allowing NAVIS to serve as a long-term experimental platform. Current development efforts therefore focus on extending reproducibility, scalability, and interoperability. One direction is navigation trace

replay and automation, which enables deterministic re-execution of recorded user trajectories for reproducible inspection and trace-driven evaluation, complementing large-scale volumetric datasets with user behavior traces such as MazeLab [15]. Another direction concerns point-cloud enhancement and upsampling, applied either offline or during playback, to allow visual inspection of higher-quality representations within the same interaction model. In parallel, GPU-efficient rendering through WebGPU integration is being explored to improve scalability, frame pacing, and robustness when handling ultra-dense point clouds, while preserving the existing buffered playback and parallel parsing architecture. Beyond point-cloud playback, future extensions include support for mesh-based representations and immersive or VR-based playback to enable richer visualization and user interaction. In addition, NAVIS is planned to support lightweight integration with external simulators, streaming frameworks, and content preparation tools (e.g., V3CTK [16] and Prism [14]), allowing streamed or simulated content to be visualized directly in the player for qualitative inspection of encoding, streaming, and adaptation trade-offs. Through these ongoing efforts, NAVIS is intended to remain a shared and evolving platform for browser-native volumetric media visualization, user behavior analysis, and immersive media research.

6 Conclusion

This paper presented NAVIS, a fully browser-native volumetric media player for interactive visualization, playback, and exploration of dynamic point-cloud video. NAVIS operates entirely within the web environment, combining WebGL-based rendering, buffered playback, adaptive parallel parsing using web workers, and integrated navigation and instrumentation to enable responsive 6DoF interaction without native dependencies. Through evaluation on multiple dynamic PLY datasets at 30 fps and 60 fps, we showed that increasing parsing parallelism significantly reduces startup delay and mitigates playback stalls, while exposing clear trade-offs between concurrency and buffer stability for high-density, high-frame-rate content. These results demonstrate that high-performance volumetric media visualization is feasible directly in the browser using standard web technologies, while remaining transparent, reproducible, and interactive. By unifying visualization, performance measurement, and user navigation within a single platform, NAVIS provides a practical foundation for experimentation, debugging, and demonstration of browser-native immersive media systems.

Acknowledgments. This research is funded by the Natural Sciences & Engineering Research Council (NSERC)-Discovery Grant RGPIN-2023-04744 and NSERC Alliance (ALLRP 586434-23)-Mitacs Accelerate (IT36792) grant.

References

- [1] 5G-MAG. 2023. RT-V3C Unity Player. <https://github.com/5G-MAG/rt-v3c-unity-player>. Accessed: 2025-03.
- [2] Abdelhak Bentaleb, May Lim, Sarra Hammoudi, Saad Harous, and Roger Zimmermann. 2025. Solutions, challenges, and opportunities in volumetric video streaming: an architectural perspective. *ACM Transactions on Multimedia Computing, Communications and Applications* 21, 7 (2025), 1–35.
- [3] Paolo Cignoni, Guido Ranzuglia, M Callieri, M Corsini, F Ganovelli, N Pietroni, M Tarini, et al. 2011. MeshLab. (2011).
- [4] Brian Danchilla. 2012. Three.js framework. In *Beginning WebGL for HTML5*. Springer, 173–203.
- [5] Eugene d'Eon, Bob Harrison, Taos Myers, and Philip A Chou. 2019. Jpeg pleno database: 8i voxelized full bodies (8ivfb v2)-a dynamic voxelized point cloud dataset.
- [6] Guillaume Gautier, Alexandre Mercat, Louis Fréneau, Mikko Pitkänen, and Jarno Vanne. 2023. UVG-VPC: Voxelized Point Cloud Dataset for Visual Volumetric Video-based Coding. In *2023 15th International Conference on Quality of Multimedia Experience (QoMEX)*, 244–247. doi:10.1109/QoMEX58391.2023.10178589
- [7] Daniel Girardeau-Montaut et al. 2016. CloudCompare. *France: EDF R&D Telecom ParisTech* 11, 5 (2016), 2016.
- [8] Jeroen van der Hooft, Maria Torres Vega, Tim Wauters, Christian Timmerer, Ali C. Begen, Filip De Turck, and Raimund Schatz. 2020. From Capturing to Rendering: Volumetric Media Delivery with Six Degrees of Freedom. *IEEE Communications Magazine* 58, 10 (2020), 49–55. doi:10.1109/MCOM.001.2000242
- [9] Lauri Ilola, Lukasz Kondrad, Sebastian Schwarz, and Ahmed Hamza. 2022. An overview of the mpeg standard for storage and transport of visual volumetric video-based coding. *Frontiers in Signal Processing* 2 (2022), 883943.
- [10] Valentin Kraft. 2019. UE4 GPU Point Cloud Renderer. https://github.com/ValentinKraft/UE4_GPUPointCloudRenderer. Accessed: 2025-03.
- [11] Zhi Liu, Qiyue Li, Xianfu Chen, Celimuge Wu, Susumu Ishihara, Jie Li, and Yusheng Ji. 2021. Point Cloud Video Streaming: Challenges and Solutions. *IEEE Network* 35, 5 (2021), 202–209. doi:10.1109/MNET.101.2000364
- [12] Pat Marion, Roland Kwitt, Brad Davis, and Michael Gschwandtner. 2012. PCL and ParaView — Connecting the dots. In *2012 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, 80–85. doi:10.1109/CVPRW.2012.6238918
- [13] MPEG Group. 2023. MPEG 3DG Renderer. <https://github.com/MPEGGroup/mpeg-3dg-renderer>. Reference implementation for MPEG 3D Graphics rendering, accessed March 2025.
- [14] Jérémy Ouellette and Abdelhak Bentaleb. 2026. Prism: A Trace-Driven Simulator for Component-Aware Adaptive Streaming of V3C Content. In *Proceedings of the 17th ACM Multimedia Systems Conference* (Hong Kong, Hong Kong) (MMSys '26). Association for Computing Machinery, New York, NY, USA, 6 pages. doi:10.1145/3793853.3799810
- [15] Jeremy Ouellette, Jashanjot Singh Sidhu, and Abdelhak Bentaleb. 2025. MazeLab: A Large-Scale Dynamic Volumetric Point Cloud Video Dataset With User Behavior Traces. In *Proceedings of the 16th ACM Multimedia Systems Conference* (Stellenbosch, South Africa) (MMSys '25). Association for Computing Machinery, New York, NY, USA, 298–304. doi:10.1145/3712676.3718345
- [16] Jérémy Ouellette, Jashanjot Singh Sidhu, and Abdelhak Bentaleb. 2026. V3CTK: An End-to-End V3C Content Preparation Toolkit for Tiled Dynamic Point Cloud Streaming. In *Proceedings of the 17th ACM Multimedia Systems Conference* (Hong Kong, Hong Kong) (MMSys '26). Association for Computing Machinery, New York, NY, USA, 6 pages. doi:10.1145/3793853.3799816
- [17] Tony Parisi. 2012. *WebGL: up and running*. " O'Reilly Media, Inc."
- [18] Nguyen Long Quang, Nguyen Thi Anh Tho, Tran Gia Minh, Tran Dai Nghia, Truong Thu Huong, and Duc V. Nguyen. 2025. Toward Optimal Viewport Adaptive Volumetric Video Streaming: Problem, Essential Tools, and Datasets. In *2025 Sixteenth International Conference on Ubiquitous and Future Networks (ICUFN)*, 262–267. doi:10.1109/ICUFN65838.2025.11170061
- [19] Sandvine. 2024. The Global Internet Phenomena Report 2024. <https://www.sandvine.com/global-internet-phenomena-report-2024>. White Paper. Accessed 20 October 2024.
- [20] Markus Schütz. 2015. *Potree: Rendering large point clouds in web browsers*. Ph.D. Dissertation. Technische Universität Wien.
- [21] Shanghong Zhao, Wei Tsang Ooi, Axel Carlier, Geraldine Morin, and Vincent Charvillat. 2013. 3D mesh preview streaming. In *Proceedings of the 4th ACM Multimedia Systems Conference* (Oslo, Norway) (MMSys '13). Association for Computing Machinery, New York, NY, USA, 178–189. doi:10.1145/2483977.2484001
- [22] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. 2018. Open3D: A modern library for 3D data processing. *arXiv preprint arXiv:1801.09847* (2018).